

Mutual Fund Analytics Platform

Task 1 — Project Setup + Data Ingestion (ETL)

Purpose. Build a modular Python ETL pipeline that loads the 10 provided mutual-fund datasets, validates and cleans them, enriches the data with fund metadata and live NAV information, and produces analysis-ready datasets.

Scope. This document records the completed Task 1 implementation and the resulting pipeline outputs. It does not claim completion of later EDA, SQL, dashboard, or advanced-analytics tasks.

1. Task 1 Deliverables

Area	Completed implementation
Project setup	Modular pipeline package with configuration, loaders, cleaners, enrichers, MFAPI client and writers.
10 CSV ingestion	All 10 raw CSV datasets load successfully with pandas.
Data validation	Missing-value, duplicate, datatype, key, date and table-specific validation checks.
Live NAV	MFAPI integration for the 40 AMFI codes in fund_master; 637 live NAV rows fetched in the completed online run.
Processed outputs	7 analysis-ready datasets written as CSV and Parquet.

2. Data Inventory

Dataset	Rows	Key purpose
Fund master	40	Scheme metadata, fund house, category, plan, risk and expense information.
NAV history	46,000	Historical daily NAV for the 40 schemes.
AUM by fund house	90	Fund-house AUM and scheme counts.
Monthly SIP inflows	48	SIP inflow, active accounts, new accounts and SIP AUM.
Category inflows	144	Monthly net inflows by fund category.
Industry folio count	21	Industry folio totals split by major asset classes.
Scheme performance	40	Returns, alpha, beta, Sharpe, Sortino, volatility and drawdown.
Investor transactions	32,778	SIP, lumpsum and redemption transactions with demographics.
Portfolio holdings	322	Equity holdings, sector weights and market values.
Benchmark indices	8,050	Daily index closing values.

3. ETL Architecture

Raw CSVs → Loaders → Cleaners/Validation → MFAPI NAV enrichment → Enrichers/joins → Writers → Processed CSV + Parquet

Modules:

- **config.py** — paths, raw-file mapping, processed output path and MFAPI endpoint.
- **loaders.py** — reads and type-casts the 10 CSV datasets.
- **cleaners.py** — table-specific validation, whitespace normalization, date handling and derived fixes.
- **mfapi_client.py** — retrieves NAV JSON from api.mfapi.in with timeout/retry handling.

- **enrichers.py** — joins NAV to fund metadata, enriches performance and transactions, and builds market summary data.
- **writers.py** — writes analysis-ready CSV and Parquet files.
- **data_ingestion.py** — orchestrates the full pipeline and supports offline execution.

4. Cleaning & Validation

- All string fields are stripped of unnecessary whitespace.
- Date fields are parsed into proper datetime values where applicable.
- Duplicate records are checked; NAV uniqueness is enforced on (amfi_code, date).
- Fund master expense ratio and exit-load ranges are validated.
- Transactions are checked for positive amounts and valid transaction types.
- Holdings are checked for approximately 100% total weight per fund, with warnings where applicable.
- Missing SIP YoY values are calculated from the available SIP inflow history.
- AMFI codes are used as the central join key across fund-related tables.

5. MFAPI Live NAV Integration

The pipeline calls the public MFAPI endpoint for each AMFI code in the fund master. The response contains dated NAV observations, which are parsed and appended/upserted into the historical NAV dataset using (amfi_code, date) as the uniqueness key. The completed online run fetched **637 live rows across 40 funds** and increased the NAV table from **46,000 to 46,637 rows**.

The client is designed to fail gracefully: network failures do not prevent the rest of the pipeline from running, and offline mode skips the live fetch entirely.

6. Output Datasets

Output	Rows	Columns	Use
funds	40	15	Clean fund master
nav	46,637	10	Historical + live NAV with fund metadata/returns
performance	40	25	Scheme performance and derived metrics
transactions	32,778	16	Cleaned and fund-enriched investor transactions
holdings	322	8	Portfolio holdings
market_summary	303	19	Monthly AUM/SIP/folio/category macro data
benchmark	1,150	8	Wide benchmark index time series

7. Run Instructions

From the project root, activate the virtual environment and run:

```
python data_ingestion.py --offline
```

Use offline mode to validate the pipeline without network access. For live NAV enrichment, run:

```
python data_ingestion.py
```

The processed datasets are written to data/processed/.

8. Task 1 Status

Status: Complete. The 10 raw datasets were successfully inspected and processed; the modular ETL pipeline runs successfully in offline mode; the online run successfully retrieved live MFAPI NAV data; and the 7 processed datasets were generated in CSV and Parquet formats.

Later project stages such as SQL database design, EDA notebooks, performance analytics, dashboard development and final deployment are intentionally outside the scope of this Task 1 document.